

05D. 파인튜닝 모델 설계 — ★미실행 · 설계안

발표 보고서 제출 요구 항목 「파인튜닝 모델 설계」의 정보이다. 현재 사용 모델은 → [05C 사용 LLM 모델.md](#) 기준일 2026-08-04

0. ★먼저 — 우리는 파인튜닝을 하지 않았다

이 문서는 **설계안**이다. 학습 실행 결과가 아니다. 이번 릴리스의 품질 향상은 전부 다른 데서 왔다.

무엇으로 올렸나	실측
임베딩 모델 교체 (ko-sroberta@128 → Arctic-ko@8192)	뒷부분 검색 MRR 0.133 → 0.317 , 잘림 83.7% → 0%
리랭커 도입 (Qwen3-Reranker-4B)	retrievable Hit@1 64.65% → 84.71%
사람 승인 OCR facts (\$7.1)	850 facts 편입 · 기존 gold rank 회귀 0
데이터 정제 (s6 포함 구조화)	인용 가능 조항 195,630/204,098 = 95.85%(01_수집데이터_및_전처리.md §3-4 정보 기준)

★발표에서 "파인튜닝 완료"라고 말하지 않는다. 이 문서의 모든 수치는 **설계 목표값**이고, 실측된 것과 구분해 읽어야 한다.

1. 목적과 비범위

1-1. 파인튜닝이 고칠 수 있는 것

생성 단계의 **형식·태도** 문제다.

문제	예
근거 밖 문장을 덧붙임	검색 근거에 없는 "보통 이런 경우는..."
기권해야 할 때 답해 버림	근거 0건인데 그럴듯한 설명 생성
인용을 문장에 연결하지 않음	근거는 있는데 어느 주장의 근거인지 안 밝힘
출력 schema 위반	verdict 를 자유문장으로 씀

1-2. ★파인튜닝이 고칠 수 없는 것 — 이게 더 중요하다

RAG 오류의 대부분은 생성 이전에 발생하고, **생성 모델을 학습시켜도 사라지지 않는다**.

실패	왜 파인튜닝으로 안 되나
검색이 조항을 못 찾음	후보에 없는 것을 생성 모델이 만들어 낼 수 없다(만들면 그게 환각이다)
잘못된 판본을 골랐음	판본 확정은 policy_version 조희 문제다
표의 행·열이 깨졌음	전처리 문제다. 학습으로 복원하면 그럴듯하게 틀린 값이 생긴다
약관 자체에 없음	없는 것은 없다. 기권이 정답이다

★"OCR 불일치를 생성 LLM 이 상식적으로 교정하게 한다"는 절대 하지 않는다. 불일치는 교정 대상이 아니라 기권 대상이다.

1-3. 그래서 착수 조건을 먼저 정한다

아래 넷을 모두 만족하기 전에는 시작하지 않는다.

- 1. 승인된 **질의-근거-답변** 데이터 $\geq 3,000$ 건
- 2. 보험사-세대-판정상태별 **holdout 분리**, 동일 문서 문장이 train/test 에 중복되지 않음
- 3. 답변의 **모든 주장**에 citation 이 연결되고 **이중 검수** 통과
- 4. base RAG 대비 **근거성 · 기권 정확도 · 인용 정확도를 동시에** 개선

현재 충족 상태

조건	지금	판정
1. 승인 QA $\geq 3,000$	승인 OCR facts 850 · 승인 QA 쌍 0	✖
2. holdout 분리	검색 평가셋 417 은 있으나 생성용 아님	✖
3. 이중 검수	프로세스 미정의	✖
4. 동시 개선 증명	baseline 측정 자체가 없음	✖

→ **착수 불가**. 이 문서는 "조건이 갖춰졌을 때 이렇게 한다"를 못 박아 두는 것이다.

2. 대상 모델과 방식

항목	값	근거
베이스	<code>google/gemma-4-E4B-it</code> (현행 생성 모델과 동일 계열)	배포 경로를 바꾸지 않기 위해
revision	<code>bb3b92e6f031fa438b409f898dd9f14f499a0cb0</code>	<code>model_registry.yaml</code> — ★고정 필수
라이선스	Gemma Terms of Use	파생 adapter 배포 전 조항 재확인 필요
방식	QLoRA (4-bit 베이스 + LoRA adapter)	12GB 단일 GPU 에서 가능한 유일한 선택
산출물	LoRA adapter (베이스 가중치 미변경)	되돌리기가 adapter 제거 한 줄

★**full fine-tuning** 을 하지 않는다. 12GB 에 안 들어가고, 되돌릴 수 없으며, 베이스 모델 업데이트를 못 따라간다.

3. 학습 데이터 규격

3-1. 입출력 형식 — 서비스 스키마를 그대로 쓴다

★**학습 형식**과 **서빙 형식**이 다르면 그 차이만큼 성능이 새어 나간다. `app/schemas/precheck.py` 의 실제 필드를 그대로 쓴다.

```
// 입력 (프롬프트로 조립)
{
  "context": {
    "insurer": "삼성화재", // ← 게이트가 확정된 것. 모델이 고르지 않는다
    "product_name": "무배당 삼성화재 다이렉트 실손의료비보험(1904.3)",
    "generation": 3, "generation_label": "3세대 (착한실손)",
    "enrolled_on": "20190501",
    "date_confidence": "exact", "parse_status": "ok"
  },
  "question": "K02.1 치료가 보장되나요?",
  "evidence": [ // ← 검색·리랭크·인용게이트를 통과한 것만
    { "clause_id": "eb778fa68b56/보통약관/제4조#6dcb03bc",
      "qualified_no": "보통약관/제4조", "title": "보상하지 않는 사항",
      "page_from": 56, "page_to": 68,
      "tier": "POLICY_CLAUSE", // ★POLICY_CLAUSE 만 학습에 넣는다
      "text": "..." }
  ],
  "output_schema": "PrecheckResult@v..."
}
```

```
// 출력 (모델이 생성하는 것)
{
  "verdict": "needs_documents", // 4단 enum 밖의 값은 schema 위반
  "abstained": false,
  "reason_code": "exception_applies",
  "message": "면책의 예외에 해당합니다. 요양급여 해당 여부 등 조건을 확인해야 합니다.",
  "citations": ["eb778fa68b56/보통약관/제4조#6dcb03bc"], // ★message 의 근거
  "abstain_reason": null
}
```

★ **verdict** 를 모델이 자유문장으로 쓰지 못하게 한다. 제약 디코딩(XGrammar 등)으로 enum-JSON 구조를 디코딩 단계에서 강제한다 — 후처리 파싱으로 고치면 그 사이에 오류가 숨는다.

3-2. 데이터 출처와 목표 건수

종류	출처	목표	지금
A. 승인 조항 QA	확정 문서 850건의 조항 → 사람이 만든 질의·답변	1,800	0
B. 승인 OCR facts QA	S7.1 승인 850 facts (자기부담금·보장한도)	600	0
C. ★기권 사례 (hard negative)	근거 0건 · 판본 모호 · 인용 미검증 상황	600	0
합		≥ 3,000	0

★C 를 별도 축으로 둔다. 정답만 학습시키면 모델은 항상 답하는 쪽으로 기운다. 기권해야 하는 상황을 같은 비중으로 넣어야 기권이 학습된다. C 의 소스는 이미 있다 — `ops.interaction_log` 의 `abstained=true` 기록과 지식캡 큐.

3-3. ★제외 — 넣으면 안 되는 것

제외 대상	왜
미승인 candidate facts (8,622건)	사람이 승인하지 않았다. 학습하면 승인 게이트가 무의미해진다
다른 판본이 섞인 근거	판본이 다르면 정답이 다르다
생성만 있고 검수 없는 데이터	모델 출력을 모델에 다시 먹이면 오류가 증폭된다
<code>tier != POLICY_CLAUSE</code> 인 근거	외부 리포트·통계는 판정 근거가 될 수 없다
<code>parse_status != "ok"</code> 문서	판정에 못 쓰는 데이터로 학습하지 않는다

3-4. ★분할 — 랜덤 70/15/15 를 쓰지 않는다

같은 약관 조항이 최대 170개 문서에 그대로 실린다(중복률 66.5%). 행 단위로 랜덤 분할하면 **train** 의 문장이 **test** 에 그대로 들어가 성능이 부풀려진다.

```
분할 키 = (document_sha256, product_line) ← 문서·상품계열 단위로 통째로 나눈다
train 70%  valid 15%  test 15%
```

추가 검사 (분할 후 반드시)

- ① `content_hash` 교집합 = 0 (같은 조항 내용이 양쪽에 없다)
- ② 보험사 12곳이 `test` 에 모두 존재
- ③ 세대 1~5 가 `test` 에 모두 존재
- ④ 기권 사례(C) 비율이 `train/test` 에서 $\pm 3\%$ 이내
- ⑤ `seed` 고정 (42) · 분할 결과 SHA-256 을 `manifest` 에 기록

★①이 0 이 아니면 학습을 시작하지 않는다. 누수가 있는 실험은 결과가 아니라 소음이다.

4. QLoRA 설정 — 못 박을 수 있는 값

★초기값이다. pilot 측정 전에는 이 값이 최적이라고 주장하지 않는다(\$4-3).

4-1. 양자화 · 어댑터

```
quantization:
  load_in_4bit: true
  bnb_4bit_quant_type: nf4          # ★fp4 아님. 정규분포 가중치에 nf4 가 맞다
  bnb_4bit_use_double_quant: true   # 양자화 상수까지 양자화 - 약 0.4bit/param 절약
  bnb_4bit_compute_dtype: bfloat16 # ★fp16 아님. 학습 중 overflow 회피

lora:
  r: 16
  lora_alpha: 32                    # alpha/r = 2.0
  lora_dropout: 0.05
  bias: none
  task_type: CAUSAL_LM
  target_modules: [q_proj, k_proj, v_proj, o_proj]
```

★ `target_modules` 를 **attention 4개로** 시작한다. MLP(`gate/up/down`)까지 넣으면 학습 파라미터가 2~3배가 되고 VRAM·시간이 늘지만, 우리 과제(형식·기권 태도)는 **표현 학습이 아니라 행동 학습**이라 attention 만으로 충분한지 먼저 본다. 부족하면 그때 넓힌다 — 넓히는 실험은 §7 의 ablation 에 넣는다.

4-2. 학습 스케줄

```
max_seq_length: 2048          # 근거 3~5건 + 질의 + 출력이 들어가는 길이
per_device_train_batch_size: 1
gradient_accumulation_steps: 16 # 유효 배치 16
gradient_checkpointing: true   # ★VRAM ↔ 속도 교환. 12GB 에서 필수
learning_rate: 2.0e-4
lr_scheduler_type: cosine
warmup_ratio: 0.03
num_train_epochs: 2           # ★3,000건 규모에서 3 이상은 과적합 위험
weight_decay: 0.0
max_grad_norm: 0.3
optim: paged_adamw_8bit       # 12GB 에서 optimizer state 압박 완화
seed: 42
```

★epoch 를 2 로 둔다. 3,000건은 작다. 작은 데이터에 오래 돌리면 **기권 문구를 통째로 외워** 근거가 있는 상황에서도 기권하게 된다.

4-3. ★pilot 전에는 확정할 수 없는 것

항목	왜 못 박을 수 없나
peak VRAM	E4B 는 per-layer embedding 구조라 공개 파라미터 수로 환산이 안 맞는다
step 당 시간 · 총 학습 시간	커널·드라이버·시퀀스 분포에 따라 달라진다
안전한 최대 max_seq_length	2048 이 OOM 없이 도는지는 측정해야 한다
최종 분할 건수	데이터가 아직 0건이다
성능 향상 폭	★baseline 조차 측정하지 않았다

pilot 절차: 데이터 200건 · 50 step 으로 먼저 돌려 peak VRAM · step/s 를 재고, 그 값으로 max_seq_length 와 gradient_accumulation_steps 를 확정한다.

5. VRAM 예산 (RTX 4070 SUPER 12GB) — ★추정

① 4-bit 베이스 가중치 = 파라미터수 × 0.5 byte × (1 + double-quant 오버헤드)

② LoRA 파라미터 = $r \times (d_{in} + d_{out}) \times \text{대상모듈수} \times \text{층수}$

③ optimizer state = ② × 2 (paged AdamW 8bit 기준)

④ 활성화값(activations) = $f(\text{batch}, \text{seq_len}, \text{hidden}, \text{층수}) \div \text{gradient_checkpointing}$

⑤ 단편화·CUDA 컨텍스트 여유 = 총합의 10~15%

구성 요소	추정	확실도
① 베이스 (E4B · NF4 · double quant)	2.5~3.0 GiB	중 — 공식 Q4 GGUF 텍스트 5.15GB 를 기준으로 역산
② + ③ LoRA + optimizer ($r=16 \cdot q/k/v/o$)	0.3~0.6 GiB	중
④ 활성화값 (seq 2048 · batch 1 · checkpointing on)	1.5~2.5 GiB	낮음 ← pilot 필요
⑤ 여유	0.6~0.9 GiB	—
합	약 5~7 GiB	12GB 안에 여유 있게 들어갈 전망

★이 표는 실측이 아니다. model_registry.yaml 의 memory_peak_mb 는 아직 null 이다. 학습을 시작할 때 pilot 로그의 torch.cuda.max_memory_allocated() 값을 여기 채워 넣는다.

★학습 중에는 OCR·검색·리랭커를 GPU 에서 내린다. 05C §6 의 「전부 상주」 예산은 추론 기준이고 학습과 동시에 성립하지 않는다.

6. 데이터 반출 제약 — ★설계의 일부다

학습 데이터에는 약관 원문 조각과 문서 청크가 들어간다.

가능	불가
로컬 GPU 상자(RTX 4070 SUPER)에서 학습	★RunPod 등 외부 GPU 로 학습 데이터 전송
공개 모델 가중치를 외부에서 받기	의료문서로 만든 LoRA 체크포인트를 외부에 두기
완전 합성 데이터로 파이프라인 시험	일부 필드만 마스크한 실제 문서

★암호화 전송·저장을 해도 GPU 메모리에서는 평문으로 처리된다. "원문을 외부로 보내지 않는다"는 제약을 충족하지 못한다.

→ 학습은 로컬 12GB 안에서 끝나야 한다. 이것이 \$2 에서 QLoRA 를 고른 이유이기도 하다.

7. 평가 프로토콜

7-1. baseline 을 먼저 고정한다

baseline = 같은 검색·리랭크·게이트 + 파인튜닝 안 한 Gemma-4-E4B

★검색 경로를 함께 바꾸지 않는다. 둘을 같이 바꾸면 무엇이 기여했는지 알 수 없다.

7-2. 지표 — 넷을 동시에 본다

지표	정의	합격 기준(안)
citation precision	인용한 조항이 실제로 그 주장을 지지하는 비율	baseline 이상, 하락 0
groundedness	답변의 각 주장이 제공된 근거로 검증되는 비율	baseline 대비 +5%p 이상
abstention precision / recall	기권해야 할 때 기권했나 / 기권하지 말아야 할 때 안 했나	recall $\geq$ 0.95, precision 하락 0
schema validity	출력이 PrecheckResult 스키마를 만족하는 비율	100% (제약 디코딩이므로)

★하나라도 회귀하면 채택하지 않는다. 특히 abstention recall 은 안전 지표다 — 여기가 떨어지면 "모르는데 답하는" 모델이 된 것이고, 다른 지표가 올라도 실패다.

7-3. 층별로 본다 — 평균만 보지 않는다

보험사 12곳별 · 세대 1~5별 · verdict 4단별
→ ★최저 구간이 baseline 보다 나쁘면 평균이 올라도 채택하지 않는다

한 보험사에서만 좋아지고 다른 곳이 나빠지면 그건 개선이 아니라 편향이다.

7-4. 표본이 작으면 구간으로 말한다

test 가 수백 건 규모라면 점추정을 단정하지 않는다. paired bootstrap 10,000회 · 95% 신뢰구간을 붙인다(리랭커 평가와 같은 방식). 구간 이 0 을 걸치면 "차이를 확인하지 못했다" 라고 적는다 — "같다"가 아니다.

8. 재현과 롤백

8-1. 재현에 필요한 것 (manifest 로 고정)

```
베이스 모델 repo + revision      bb3b92e6...
학습 데이터 SHA-256              (분할 후 train/valid/test 각각)
분할 seed                        42
QLoRA 설정 파일 SHA-256
라이브러리 버전                  transformers · peft · bitsandbytes · torch
GPU · 드라이버 · CUDA
adapter 체크포인트 SHA-256
평가 결과 JSON SHA-256
git commit + dirty 해시
```

★ 모델 산출물도 데이터 산출물과 같은 규칙을 따른다 — 경로에 버전을 박아 덮어쓰지 않고 공존시킨다.

8-2. 배포 게이트

```
pilot → 학습 → 평가($7) → ★사람 검수 → model_registry 에 새 프로필 추가
                                   artifact_sha256 · verified_at 채움
                                   → A/B 로 일부 트래픽 → 회귀 없으면 전환
```

★ artifact_sha256 이나 verified_at 이 비어 있으면 배포하지 않는다. 지금 local_gemma4_e4b 가 그 상태이며, 그래서 "승인된 모델"이라고 부르지 않는다.

8-3. 롤백

adapter 방식이라 되돌리기가 싸다.

1. .env 의 LLM_PROVIDER / LOCAL_MODEL 을 base 프로필로 되돌린다
2. adapter 를 로드하지 않는다 (베이스 가중치는 애초에 안 바뀌었다)
3. 판정 결과는 영향받지 않는다 — ★판정은 규칙엔진과 인용 게이트가 하고
LLM 은 설명만 생성하기 때문이다(05C §2-3)

★이 마지막 줄이 이 설계의 안전장치다. 생성 모델이 나빠져도 잘못된 보장 판정으로 이어지지 않는다.

9. 리랭커 파인튜닝 — ★지금은 하지 않는다

왜	근거
zero-shot 이 이미 충분히 개선됐다	dense 64.65% → 4B 리랭크 84.71%
학습보다 먼저 고칠 것이 있다	★gold 불일치 93건(S5 평가 ID ↔ S6 delivery). 이걸 두고 학습하면 틀린 정답을 학습한다
hard negative 를 모을 로그가 없다	운영 질의-클릭-검수 로그가 아직 안 쌓였다

착수 조건이 되면: query - positive - hard negative triplet 으로 LoRA 또는 distillation. 비교는 base Qwen3 대비 **MRR·Hit@1 + 보험사/세대별 최저 성능**을 함께 본다.

10. 임베딩 파인튜닝 — ★더 뒤다

임베딩을 바꾸면 전량 재색인(약 6시간) 이고, 두 벡터 공간이 섞이면 검색이 조용히 망가진다. 현재 모델조차 예비 후보 축소 단계에 있다(05C §3-4) — 선정을 마치는 것이 먼저다.

11. 이 설계안이 주장하지 않는 것

1. "파인튜닝하면 정확도가 오른다" — 검증하지 않았다. §7 은 검증 방법이지 결과가 아니다.
 2. "\$4 값이 최적이다" — pilot 전 초기값이다.
 3. "\$5 VRAM 이 실측이다" — 추정이다. `memory_peak_mb` 는 `null` 이다.
 4. "3,000건이면 충분하다" — 착수 하한이지 충분 조건이 아니다.
 5. "지금 시작할 수 있다" — §1-3 의 네 조건이 전부 미충족이다.
-

참조

- 현재 사용 모델 측정: [05C 사용 LLM 모델.md](#)
- 출력 스키마: `app/schemas/precheck.py` · `app/core/domain/precheck_result.py`
- 모델 레지스트리 규칙: `model_registry.yaml` · `tests/` REQ-LLM-REG-01